

HALT: Human-Aware Local Triggering

A Lightweight Machine Vision Application for Safety during Human-Robot Interaction

ISE 572 Final Project

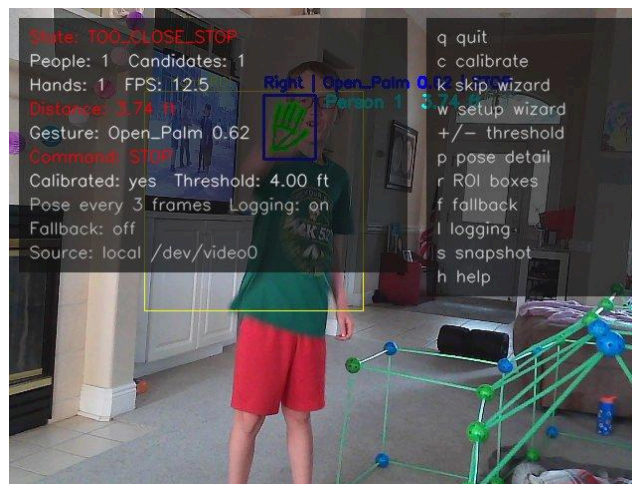
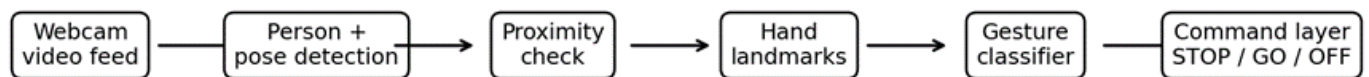
Matt Carrell and Demetrios Petrou

Dimension	A0	A1	A2	A3	A4	A5
Ideation	Yes					
Data Decisions					Yes	
Coding				Yes		
Debugging			Yes			
Interpretation				Yes		
Writing				Yes		

A. Executive Summary

HALT: Human-Aware Local Triggering is a lightweight machine vision system designed to improve safety around industrial robots and automated machines. The goal of this project was to build a low-cost, edge-based add-on that can watch for nearby human operators, estimate whether they are too close to dangerous equipment, recognize simple hand signals, and convert those observations into machine states such as CLEAR, HUMAN_NEARBY, GO, STOP, or TOO_CLOSE_STOP.

In many industrial settings, fast-moving equipment can create safety risks when people work nearby. Traditional safety systems such as fixed barriers, emergency stop buttons, or industrial sensors are important, but they may not always fit smaller operations, flexible workcells, training labs, mobile robots, or lower-budget automation projects. HALT addresses that gap by using a standard webcam, an Intel NUC edge computer, and lightweight machine learning models that run locally without cloud dependence.



The final custom gesture model was trained on 1,515 labeled hand landmark samples across three classes: no signal, thumbs up, and palm open. It reached 97.8% test accuracy and a macro-F1 score of 0.978. Most importantly, the model did not miss any intentional thumbs up or palm open gestures in the test set. The only observed errors were no signal samples being mistaken for a gesture, which is a safer failure mode than missing a STOP command. The system also includes proximity logic, so a GO command can still be blocked if a person is too close to the machine.

The final HALT application worked as a full local demo system. In manual runtime testing, the proximity alarm correctly triggered at the 6 ft threshold every time it was tested, both with one tester and with two testers present. A reviewed CSV event log also showed successful live operation of the full pipeline, including CLEAR, HUMAN_NEARBY, GO, STOP, and TOO_CLOSE_STOP states, local webcam operation, saved snapshots, and model switching between the custom .pkl classifier and the MediaPipe baseline. Overall, HALT shows that a practical edge-based awareness layer can be built with common hardware and lightweight machine learning. It is not meant to replace certified industrial safety systems, but it can support them by adding local human awareness and fallback decision logic.

B. Problem Statement & Industry 4.0 Context

Industrial robots and automated machines are becoming more common in factories, labs, warehouses, and training environments. These machines can improve speed, quality, and productivity, but they also create safety concerns when people work near them. A robot arm, conveyor, cutting machine, mobile robot, or other fast-moving equipment may not always know when a person has entered a dangerous area. In a perfect environment, the operator would communicate clearly for the machine to turn off through buttons, control panels, networked systems, or verbal commands. In the real world, that is not always reliable, and in emergency situations humans could simply forget.

Factory environments can be loud, busy, and distracting. Verbal communication may be difficult because of machine noise, hearing protection, alarms, etc. Physical controls may require the operator to walk closer to the machine to physically shut it off, which can increase risk during an emergency. Networked control systems can also be useful, but they depend on the machine being connected and the communication system staying online. If that network connection fails, the safety and command system should still have a local fallback that can respond quickly.

This project addresses that problem by creating HALT: Human-Aware Local Triggering, a low-cost edge machine vision add-on for robots and industrial equipment. The goal is to give a machine a basic local awareness layer. HALT uses a camera and an edge computer to detect when a person is nearby, estimate whether the person may be in a risk zone, and recognize simple hand commands such as STOP and GO. The system is designed to run locally, without depending on cloud processing or an outside network connection. This makes it useful as an offline backup or as an added safety layer for smaller automation setups that may not have expensive industrial sensing systems.

The key idea is simple: if a person gets too close to dangerous equipment, the machine should be able to slow down, stop, or block a GO command. If the operator gives a clear hand signal, the system should recognize it and pass that command to the machine. This would not replace certified industrial safety systems, but it could add another layer of awareness and communication.

This connects directly to Industry 4.0 because modern smart manufacturing depends on many connected sensors, edge devices, and machines working together to create a more complete view of what is happening on the factory floor. HALT fits into that idea as an edge sensor and decision layer. Instead of only collecting machine data, it collects human presence, body position, hand landmark, gesture, confidence, and command data. That data can be used immediately for local machine response, and it can also feed larger Industry 4.0 systems later for dashboards, safety analysis, process improvement, and training.

The success criteria for this project were to build a working local computer vision pipeline, detect a nearby human operator, classify simple gesture commands, run fast enough for real-time use on edge hardware, and log the data in a way that supports later analysis. The system needed to be simple enough to run on common hardware, but strong enough to show how edge machine learning can improve human-machine interaction. This collects clean landmark data, labeling gestures clearly, and evaluating whether the model can recognize safety-related commands reliably.

C. Data Pipeline & Preprocessing

1. Data Sources and Collection Methods

Training data was collected using `src/collect_data.py`, a purpose-built interactive labeling tool that streams a live webcam feed and records hand landmark rows on demand. We built the data collection process around the same camera pipeline used by the application, so the model training work stayed tied to the final live demo instead of becoming a separate notebook-only task.

The project needed to support two different development environments. For the WSL2 setup, the camera feed was routed from a Windows host to WSL2 over a local TCP socket, with `capture_windows.py` running on the Windows side and `src/capture.py` receiving frames inside WSL2. For the Intel NUC setup, the final application uses direct local webcam capture through OpenCV and V4L2. The final script tries local cameras first, normalizes frames to 640x480, and can fall back to a TCP/JPEG stream if the local camera is not available.

During a data collection session, the operator positions their hand to show the target gesture and presses a label key: 1 for `no_signal`, 2 for `thumbs_up`, or 3 for `palm_open`. On each keypress, MediaPipe Hand Landmarker checks whether at least one hand is detected in the frame. If a hand is detected, the 21 normalized landmarks for that hand are appended as one row to `model/data/landmarks.csv`. If no hand is detected, the keypress is ignored. A live HUD displays running counts per class, making it easy to monitor class balance during collection.

This workflow made data collection fast and kept the model tied directly to the final runtime application. In the live HALT system, the custom `.pkl` model uses the same landmark structure at runtime: 21 hand landmarks converted into 63 flattened features.

2. Dataset Description

The dataset is stored in `model/data/landmarks.csv` and contains 1,515 samples across three classes. Each row represents one hand detection and consists of 64 columns: one label column and 63 feature columns encoding the (x, y, z) coordinates of all 21 MediaPipe hand landmarks (`x0-x20`, `y0-y20`, `z0-z20`). The class distribution is shown in Figure 1 and is nearly perfectly balanced: `no_signal` (506), `thumbs_up` (499), and `palm_open` (510), each representing approximately 33% of the dataset. This balance was maintained by monitoring the HUD count totals during collection and labeling until all classes were equalized.

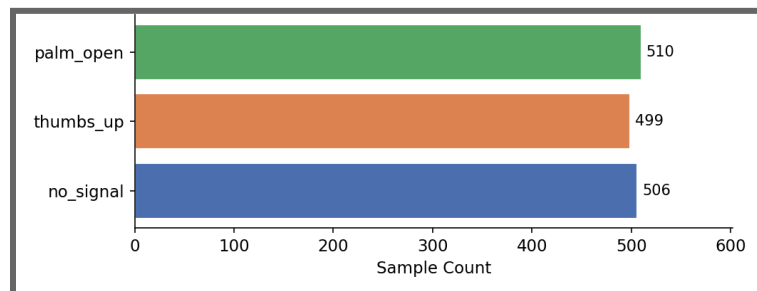


Figure 1: Class Distribution in landmarks.csv

3. Data Quality

No significant data quality issues were encountered. The dataset contains zero missing values. MediaPipe's Hand Landmarker only returns landmark data when a hand is detected with sufficient confidence, so

low-quality or ambiguous detections are filtered out before reaching the collection script. No duplicate removal or outlier filtering was applied.

4. Feature Engineering

MediaPipe's Hand Landmarker outputs coordinates that are already normalized: x and y are scaled relative to the frame dimensions and fall in the range $[0, 1]$, while z represents depth relative to the wrist landmark and is constrained to a small range near zero. Because this normalization is handled by the sensor pipeline, the only preprocessing applied to the collected landmarks was rounding the coordinates to six decimal places for storage. The 63 raw normalized coordinates are passed directly to the model with no further transformation. Feature value distributions are shown in Figure 2, confirming compact and consistent ranges across all three axes. This tight range is of course due to the bias of having the hand centered near the center of the frame during data collection, and this is how the robot would hopefully be framing a real operator's hand in a real situation. Figure 3 visualizes the mean landmark positions per class, illustrating the geometric differences that separate the three gestures. This serves as a successful sanity check on the data as the graphs represent a 2D view of the hand landmark samples. No signal generally should correlate to a more closed hand and thus clumped landmarks, thumbs up should correlate to clumped landmarks underneath a sparser group of landmarks representing the thumb, and palm open should show a more even but vertical spread of landmarks. All these assumptions are confirmed by Figure 3.

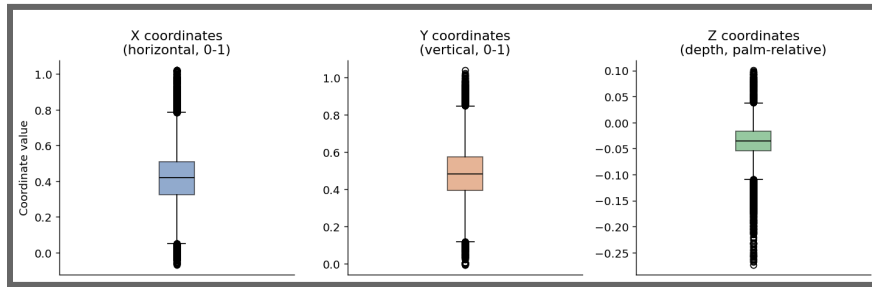


Figure 2: MediaPipe Landmark Coordinate Distributions

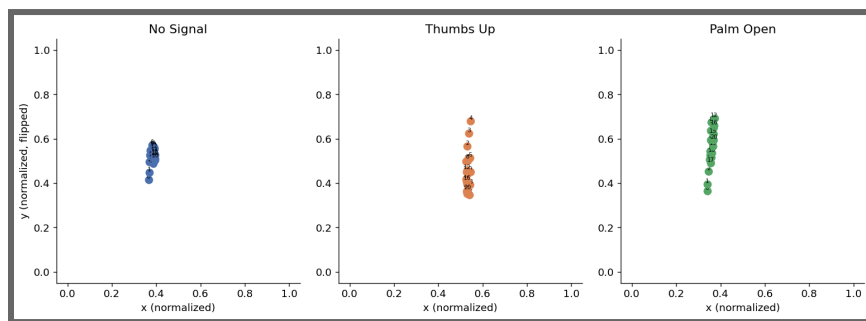


Figure 3: Mean Landmark Positions per Class

5. Train/Validation/Test Split

The dataset was partitioned using a stratified 70/15/15 train/validation/test split. Stratification was applied to preserve the near-equal class balance in every partition, as shown in Figure 4. A fixed `random_state=42` was used so that the split is fully reproducible across runs. The resulting split sizes are: train (1,060), validation (227), and test (228). The validation set was used to monitor training convergence, and the test set was held out entirely until final evaluation.

Figure 5 shows a PCA projection of the training set onto three principal components. The three gesture classes roughly form distinct clusters, providing evidence that the feature space is separable and motivating the use of a simple fully-connected network over more complex architectures.

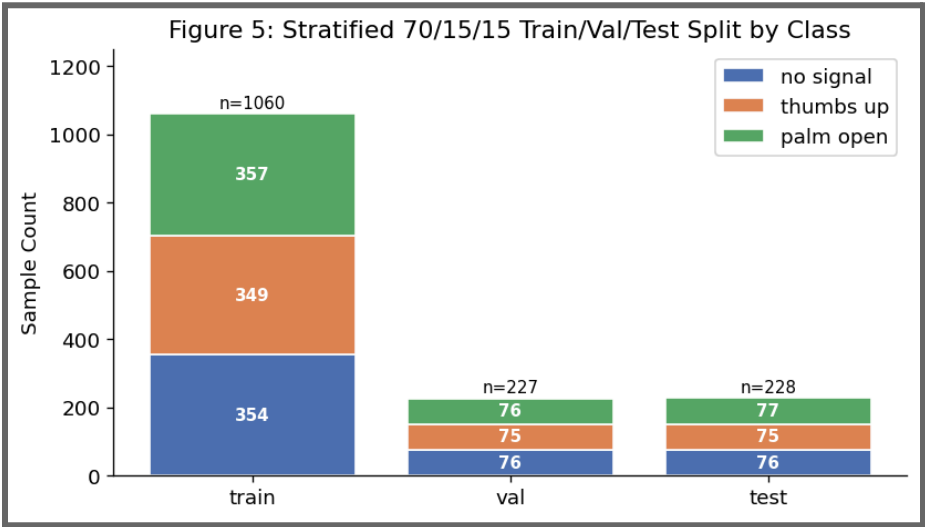


Figure 4: Stratified 70/15/15 Train/Val/Test Split by Class

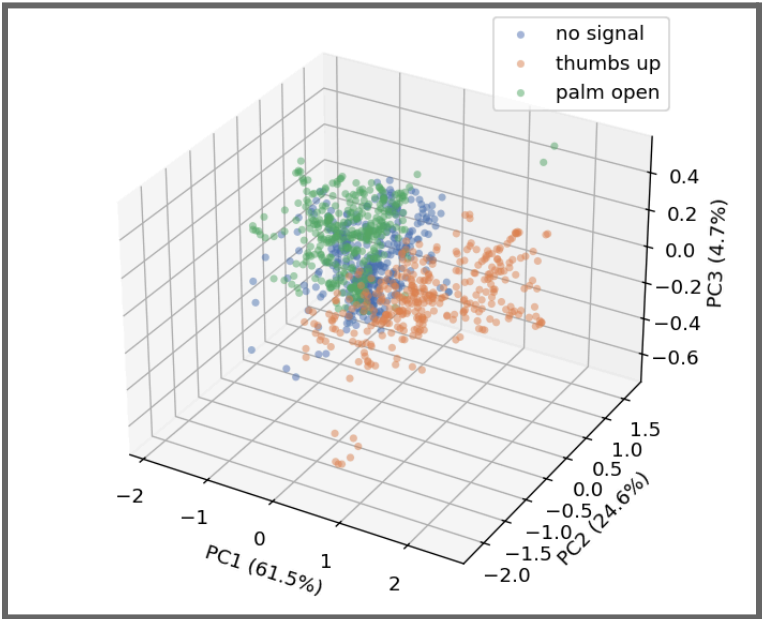


Figure 5: PCA 3D Projection of Training Set

D. Model Development & Implementation

1. Algorithm Selection and Justification

The core gesture classification task was formulated as a 3-class supervised learning problem over a flat 63-dimensional feature vector. A Multilayer Perceptron (MLP) from scikit-learn (MLPClassifier) was selected as the primary model for several reasons. First, the PCA projection in Figure 5 shows that the gesture classes are well-separated in feature space, suggesting a shallow fully-connected network is sufficient. Second, the flat vector input has no spatial grid or temporal structure that

would benefit from a CNN or RNN. Third, the scikit-learn implementation integrates directly with the existing NumPy/pandas data pipeline and avoids introducing a deep learning framework dependency – an important consideration for deployment on the Intel NUC edge machine, where inference must run in real time.

For comparison purposes, MediaPipe's pre-trained GestureRecognizer was used as a baseline. This model performs visual inference on full frames and requires no custom training data. In live testing, the custom MLP performed visually on par with the MediaPipe baseline while producing substantially higher classification confidence for positive gestures (see Section E). Additionally, MediaPipe's PoseDetector was used for proximity detection, estimating the operator's distance from the camera using a pinhole camera model to trigger the machine OFF-signal when the operator approaches within a distance threshold that is set in the demo UI.

2. Model Architecture

The MLP consists of an input layer of 63 features, two hidden layers of sizes 128 and 64 with ReLU activations, and a 3-class softmax output layer. The decreasing layer width encourages the network to learn progressively more compact gesture representations before the final classification. The model contains 16,643 trainable parameters, which is lightweight enough for sub-millisecond inference on a standard CPU.

3. Training Process

The model was trained using the Adam optimizer with scikit-learn's default learning rate schedule. A maximum of 500 iterations was set as an upper bound, and using all the features allowed training to converge in 202 iterations with a final cross-entropy loss of 0.069, down from 1.097 at initialization – a 94% reduction. The training loss curve is shown in Figure 6.

No extensive hyperparameter search was conducted. The training process quickly produced strong results on the validation set that were on par with the MediaPipe baseline, making further tuning unnecessary. Model weights and evaluation metrics for each training run are saved to a timestamped checkpoint directory (model/checkpoints/[RUN-ID]/) containing the serialized model (.pkl) and a metrics.json file with the full loss curve and per-class performance breakdown. The checkpoints/ folder is not included in our repository, but our chosen custom model is located at model/gesture_classifier.pkl.

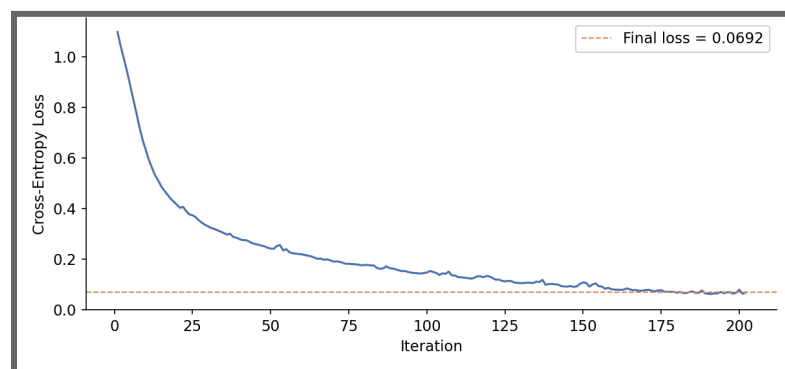


Figure 6: Training Loss Curve

4. Implementation Challenges

The final application required several engineering steps beyond training the classifier. The Intel NUC version was built around local USB webcam capture through V4L2, using MJPG video at 640x480 and 30 FPS. If a local camera is unavailable, the script can fall back to a TCP/JPEG stream. All incoming frames are normalized to 640x480 so a higher-resolution camera feed does not overload the edge computer.

Early examples using `mediapipe.solutions` did not fit cleanly with the newer MediaPipe install, so the project used the newer MediaPipe Tasks API and handled all drawing manually with OpenCV. That included pose overlays, hand overlays, labels, UI buttons, and demo text.

A key optimization was wrist-guided hand and gesture processing. Instead of running expensive full-frame hand detection continuously, the system first runs Pose Landmarker to find people and locate visible wrists and elbows. If a wrist is raised above the elbow, the script creates a 220 x 220 wrist-centered region of interest and runs hand or gesture processing only on that crop. This is much more efficient and better matches the intended use case, where the system is mainly looking for deliberate raised-hand signals.

One major implementation bug came from processing multiple wrist regions in one frame. MediaPipe VIDEO mode requires timestamps to increase every time a model is called. When multiple wrist ROIs reused the same timestamp, MediaPipe produced the error: Input timestamp must be monotonically increasing. This was fixed with a `TimestampTracker` class so pose, hand, and gesture calls each receive their own increasing timestamp sequence. This made multi-hand and multi-person processing stable.

The final script supports both the custom `.pkl` classifier and the MediaPipe `.task` gesture recognizer, and the user can toggle between them live. It also supports up to two people and up to four candidate hands at once, which made it possible to test how the edge device handled higher hand-processing loads.

E. Results & Evaluation

1. Model Performance Metrics

The custom MLP was evaluated on the held-out test set (228 samples) after training. Overall results are summarized in Figure 7 and per-class results in Figure 8. The model achieves 97.8% test accuracy with a macro-F1 of 0.978. The small gap between training and test accuracy (0.5 percentage points) indicates minimal overfitting. The validation accuracy remained slightly lower (94.7%), and this behavior was confirmed to persist even with different `random_state` values at the train/test split function calls. Our conclusion is this small difference is likely just statistical noise due to an outlier that keeps landing in the validation set.

Per-class metrics on the test set reveal a consistent pattern: `no_signal` is the hardest class to recall, while `thumbs_up` and `palm_open` are the hardest to be precise about. On the test set, `no_signal` achieves perfect precision (1.000) but the lowest recall (0.934), meaning all five misclassifications are `no_signal` samples the model mistook for a positive gesture and never the reverse. Both `thumbs_up` and `palm_open` achieve perfect recall (1.000), confirming that no deliberate gesture was ever missed, though their precision is slightly lower (0.987 and 0.951 respectively) because a small number of `no_signal` samples were pulled across their decision boundaries. This is a good look for our custom model as it indicates that the worst the model can do is perform a false alarm. The bad part is that this false alarm can be for ON signals, but even if a false alarm for ON is triggered when the user is too close, the distance threshold logic will not allow the machine to power on.

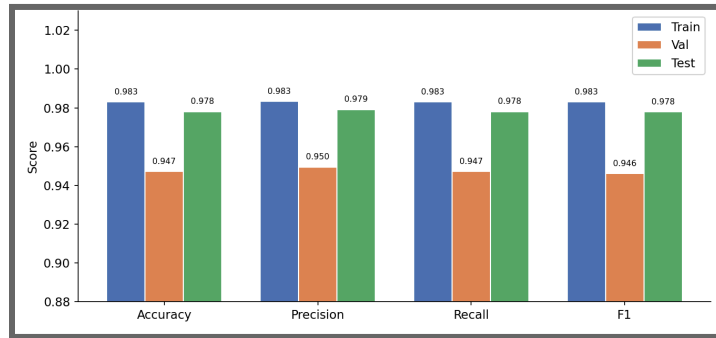


Figure 7: MLP Classifier Train/Val/Test Performance

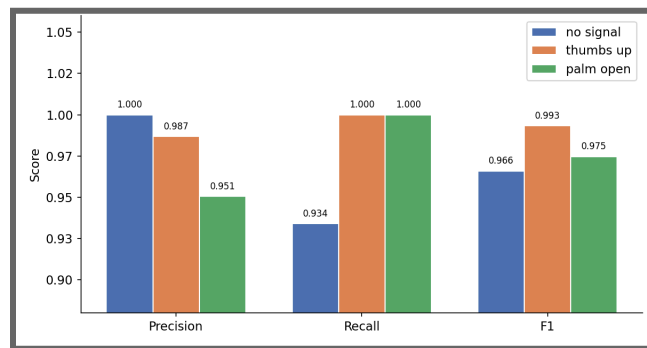


Figure 8: Per-Class Precision/Recall/F1 on Test Set

The proximity alarm testing was looser than the formal held-out gesture test set because it focused on the integrated application rather than labeled classifier evaluation. Early testing was mostly manual and qualitative because CSV logging had not yet been added to the project. Structured CSV logging was added later, so the final report uses both manual testing observations and a reviewed CSV event log.

Once the system was calibrated, the proximity logic was tested repeatedly with a single user and then with multiple users. The tester then moved toward the system in 2 ft increments, going from 10 ft to 8 ft to 6 ft to 4 ft, and then back out in the same increments. At 6 ft, the system recognized the threshold correctly every time. This single-user proximity test was conducted 8 times across multiple code iterations. A two-user version of the same test was conducted 4 times. In all of these runs, the proximity alarm worked successfully. The system correctly entered TOO_CLOSE_STOP every time the tester crossed the threshold, and once inside the threshold it continued to show TOO_CLOSE_STOP no matter what gesture was given. In other words, the distance rule consistently stayed above the gesture rule.

Reviewed CSV event logs contained 890 rows across two sessions, covering roughly 7,800 processed frames. Every row came from the local webcam path, local /dev/video0, confirming that the demo ran on the intended edge camera setup. The median logged FPS was about 15.1 FPS, which was usable for a classroom prototype while still drawing overlays, logging states, and saving snapshots.

The strongest runtime result was the distance safety behavior. The STOP threshold was set to 6.0 ft, and all TOO_CLOSE_STOP rows in the CSV logs occurred below that threshold, with estimated distances ranging from 4.39 ft to 5.97 ft matching what was seen during live testing. There were zero GO states below the 6.0 ft threshold. This directly supports the main design goal of HALT: even if a gesture suggests GO, the distance-based safety rule can still block it when the operator is too close.

2. Comparison with Baseline

The MediaPipe Gesture Recognizer served as the baseline comparison. This is not a perfectly matched comparison, because MediaPipe is a more general gesture model while the custom .pkl model was trained only for the HALT task labels: no_signal, thumbs_up, and palm_open. Still, it is a valuable comparison because it shows how a small custom model performs next to a polished existing solution.

In the larger comparison log, the system recorded 26,452 live inference frames, including 19,354 MediaPipe Gesture Recognizer frames and 7,098 custom MLP frames. Because the live frames were not manually labeled, this log was not used as a strict live accuracy test. Instead, it was used as a runtime comparison tool. The custom MLP produced noticeably stronger confidence values for the positive target gestures. For thumbs_up and palm_open, the custom model's confidence scores were more concentrated near 1.0, while the MediaPipe model's scores were broader and generally lower.

In subjective live testing, both models were workable for the simple STOP/GO demo goal, and the fact that the custom model was comparable at all is impressive given that it was trained from a small project-specific dataset. MediaPipe remains the more general recognizer, but the custom model was tuned to the exact command set needed for HALT, which likely helped it perform so well on those classes.

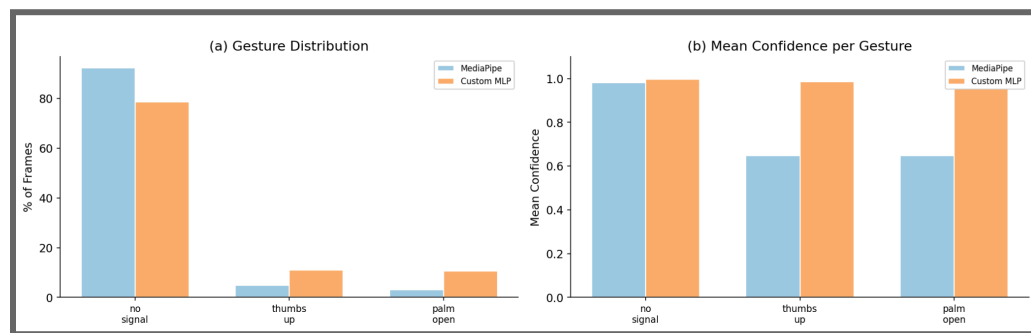


Figure 9: Baseline (MediaPipe) vs. Custom MLP Live Inference Comparison

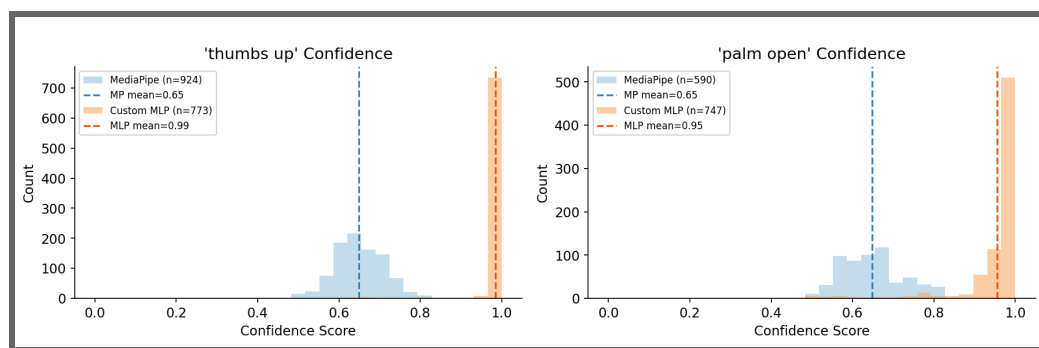


Figure 10: Confidence Distribution Comparison for Positive Gestures Only

3. Visualizations

The test set confusion matrix is shown in Figure 11. All five misclassifications fall in the no_signal row since the model never incorrectly classifies a positive gesture in our evaluation. The training loss curve in Figure 6 shows smooth convergence over 202 iterations. The softmax probability breakdown for each misclassified sample is shown in Figure 12. This reinforces the point from before that all misclassifications

by this model would generally be false alarms on actual hand signals, and mainly overzealously predicting STOP when there is no signal. As we have mentioned, it is only a few times and the better scenario than if the model missed some STOP signals.

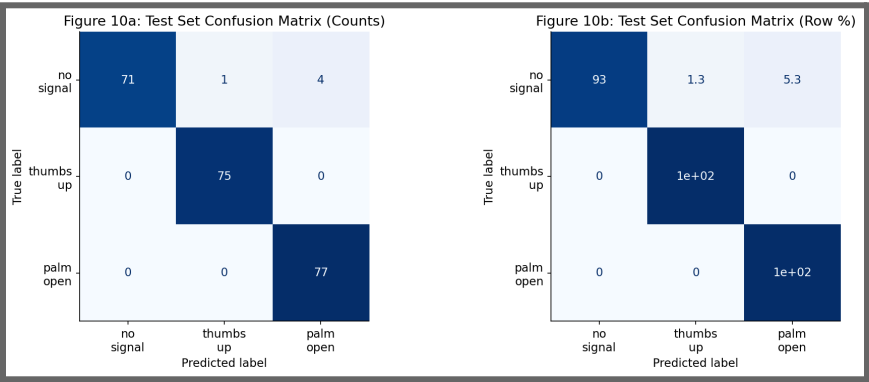


Figure 11: Test Set Confusion Matrices by Count and Percentage

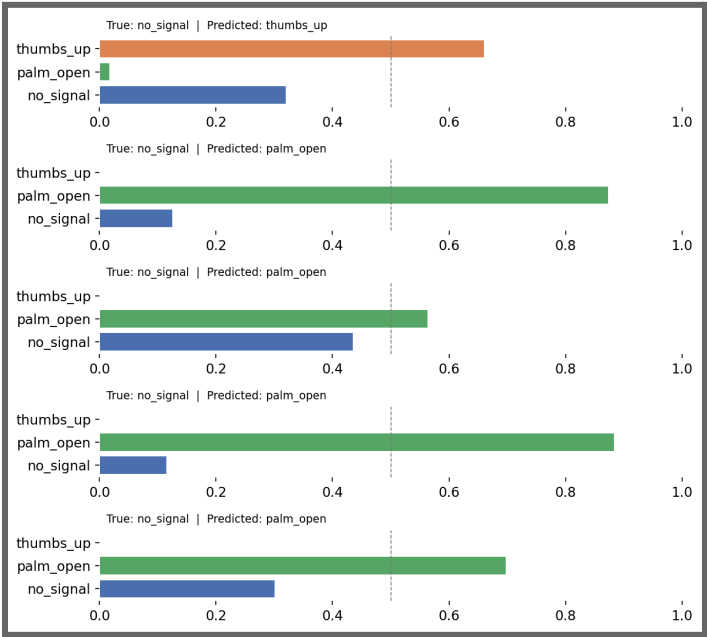
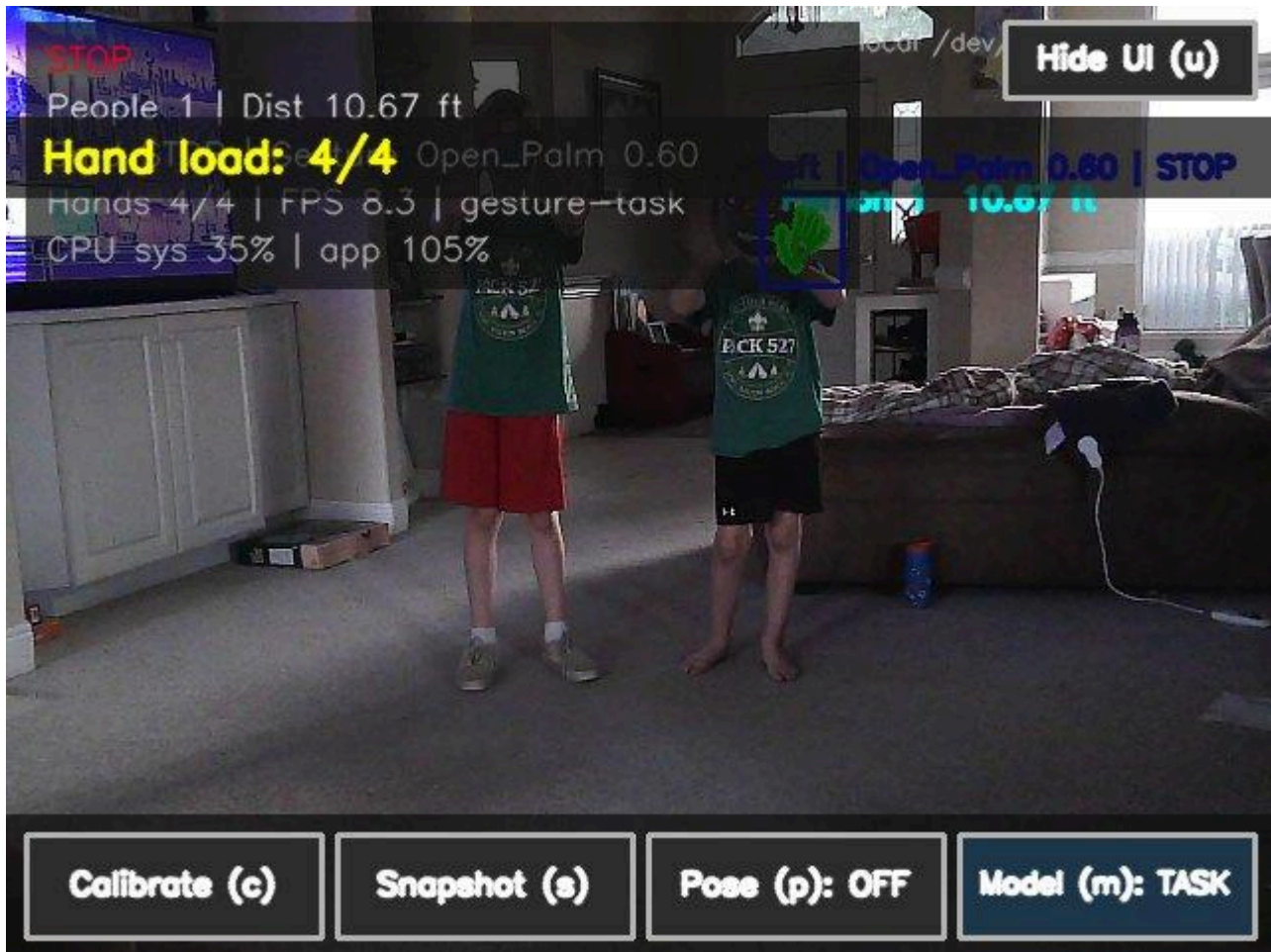


Figure 12: Softmax Probabilities for Misclassified Test Samples

The runtime results give one more layer of error analysis. Since GO never appeared below the distance threshold in the reviewed CSV, the proximity logic appears to protect against one of the more concerning false activation cases. Even if a gesture model produces a GO signal in error, the final robot-state logic can still force TOO_CLOSE_STOP.



4. Error Analysis

All five test-set errors are instances of `no_signal` being misclassified as `palm_open` (four cases) or `thumbs_up` (one case). This is consistent with the structural ambiguity of the `no_signal` class: because it is a catch-all for any non-deliberate posture, relaxed hand positions can produce landmark geometries that fall near the decision boundary of `palm_open`.

Importantly, neither `thumbs_up` nor `palm_open` was ever misclassified on the test set. Both achieve perfect recall (1.000). This is the most safety-critical result: false negatives on the STOP signal are the highest-risk failure mode, and the model produces none.

5. Discussion in Business/Safety Terms

In an industrial co-bot or machine environment, not all model mistakes carry the same cost. A missed STOP command could allow a machine to continue operating when an operator intends to halt it. A false STOP may interrupt the process, but it is usually safer than a missed STOP. The custom MLP's perfect `palm_open` recall on the held-out test set directly addresses the most safety-critical gesture case.

The proximity sensing layer adds another safety check. In manual testing, the proximity alarm triggered successfully every time with one tester and with two testers. In the reviewed CSV, `T00_CLOSE_STOP` only

occurred below the 6 ft threshold, and GO was never allowed below that threshold. This supports the main project idea: HALT should not treat gesture recognition as the only source of truth. Instead, it should combine gesture intent with human location.

The comparison results are also encouraging. MediaPipe Gesture Recognizer is a strong general-purpose baseline, so the custom model being subjectively comparable for the limited STOP/GO task is a meaningful result. It suggests that a small custom model can be very useful when the command set is narrow, the feature structure is clean, and the model is integrated into a practical edge pipeline.

For a classroom, lab, or prototype business case, this means HALT could provide a low-cost awareness layer around robots or machines. It could help smaller labs or flexible workcells collect useful human-machine interaction data, test safety logic, and demonstrate edge-based Industry 4.0 concepts without cloud dependence. It should not be presented as a replacement for certified safety systems, but it is a practical example of how local ML sensors can support safer machine behavior.

F. System Design & Deployment Considerations

HALT is designed as a local edge add-on that can sit near a robot, machine, workcell, classroom demo, or training setup. The main hardware requirement is a camera pointed toward the operator area and a small edge computer, such as the Intel NUC used in this project. The system does not need cloud processing to make its basic decisions. This is important because the most valuable part of HALT is its ability to keep working locally even if a larger network, dashboard, or communication system is unavailable.

In practice, the system continuously processes the live camera feed. The first layer detects whether a person is present. The second layer estimates whether that person is close enough to be in a possible danger zone. The third layer looks for simple hand gestures. The system then combines those signals into robot states: CLEAR, HUMAN_NEARBY, GO, STOP, or TOO_CLOSE_STOP.

The state priority keeps distance safety above gesture commands. If no person is visible, the system reports CLEAR. If a calibrated person is closer than the STOP threshold, the state becomes TOO_CLOSE_STOP. If the winning gesture is STOP, the system reports STOP. If the winning gesture is GO, the system reports GO only if the distance rule does not block it. Otherwise, the system reports HUMAN_NEARBY. This is important because a GO gesture alone should not automatically mean the machine is safe to run. The machine needs to know both what the person is signaling and whether the person is in a safe position.

Distance is estimated using a simple pinhole-style approximation. The default setup uses a 5 ft 10 in user height, a 10 ft calibration distance, and a 6 ft too-close threshold. In the demo, the operator marks a floor spot 10 ft from the camera, stands there with the full body visible, and gives a thumbs-up. The system calibrates when it sees both a person and a GO gesture. This is not true depth sensing, so if the camera moves, calibration should be repeated.

The working demo includes a simple operator interface. Instead of using a heavier GUI framework, the final script uses OpenCV buttons and keyboard controls. The user can calibrate, take a snapshot, toggle pose overlay, switch between the custom .pkl model and the MediaPipe .task model, hide the UI, show ROI boxes, toggle logging, adjust the STOP threshold, and increase hand candidate load. The clean view mode hides the extra HUD and bottom buttons while keeping the person, pose, hand, and gesture overlays visible. That made the demo easier to present because the screen was less cluttered but still showed the tracking results.

The system also creates useful data for larger Industry 4.0 systems. Even if the first use is simply local STOP and GO logic, HALT can log timestamps, gestures, model confidence, proximity state, command outputs, FPS, and snapshot paths. Over time, that data could help teams understand when people enter risk zones, how often STOP commands occur, where false alarms happen, and whether operators are using the system correctly.

The computational requirements are intentionally modest. The custom model is small, and most of the processing load comes from live video, pose tracking, hand landmarking, and drawing overlays. The hand-load test lets the user increase candidate hands from one to four and watch system performance. In basic testing, the system remained usable on the Intel NUC, and the reviewed runtime log showed a median FPS of about 15.1 while the application was running locally with overlays and logging.

The main limitations are the small dataset, controlled test environment, limited gesture vocabulary, and approximate distance logic. Gesture recognition is sensitive to lighting, hand angle, motion blur, occlusion, and whether the live camera setup matches the training data. A stronger deployment would need more samples from different people, gloves, lighting conditions, hand sizes, backgrounds, distances, and camera angles. It should also include confidence thresholds, multi-frame confirmation, startup health checks, and safe failure behavior. If the camera fails or the model cannot load, the safest behavior is to avoid issuing a GO command.

A better future test mode would also improve evaluation. The current event log proves that the application worked end-to-end, but it does not include ground-truth labels for every live frame. A guided scenario mode could log the expected gesture, expected distance zone, expected command, and expected robot state for each trial. That would allow direct live comparison between the custom .pkl classifier and the MediaPipe .task recognizer on the same frames.

Conclusion & Future Work

HALT: Human-Aware Local Triggering shows that a low-cost edge vision system can recognize nearby people and simple operator gestures around industrial machines. The main goal was not to build a certified industrial safety product. The goal was to prove that a practical, local, machine-learning-based awareness layer could detect human presence, recognize STOP and GO gestures, estimate basic proximity, and produce command states that could support safer human-machine interaction.

The project was successful as a working proof-of-concept. The final custom gesture model reached 97.8% test accuracy and a macro-F1 score of 0.978 on the held-out test set. It did not miss any intentional thumbs up or palm open gestures. The few errors came from no signal samples being classified as a gesture, which is not perfect, but is a better failure mode than missing a STOP command. The added proximity logic also helps prevent a GO command from being accepted when a person is too close to the machine.

The runtime application also showed that the system worked beyond a training script. It opened the local NUC camera, processed live frames, switched between the custom and MediaPipe gesture models, produced robot states, enforced the 6 ft STOP threshold, saved snapshots, and wrote event logs. The custom model was not just a theoretical classifier sitting in a file. It was integrated into the full application path, and its behavior was comparable enough to the MediaPipe baseline to be impressive for a small custom model trained from our own data.

Several lessons came from the project. First, the pipeline needs to be simple enough to run reliably in real time. A model that performs well but slows down the camera feed is not useful for this type of application. Second, the no signal class is difficult because it is not one clear gesture. It is a catch-all category for everything that is not a deliberate command. Third, live video systems require practical troubleshooting that does not always appear in model metrics. Camera access, frame rate, lighting, user interface overlays, MediaPipe timestamps, and model callbacks all affect whether the system feels usable.

Future work should focus on making HALT more reliable in real-world conditions. The dataset should be expanded with more users, more environments, more hand positions, gloves, varied lighting, different camera angles, and longer live sessions. The system should also measure false triggers, missed commands, latency, and user comfort over time. More gestures could be added, but only if they are easy to remember and visually distinct. For safety-related use, fewer clear commands are better than many confusing ones.

Another future improvement would be a stronger setup and deployment process. A calibration wizard could help set the camera position and distance threshold. Confidence rules could require the same gesture to appear for several frames before a command is sent. Future versions could also support multiple people and multiple hands while keeping the command logic simple and safe.

Overall, HALT demonstrates a realistic Industry 4.0 concept: a small edge device can watch the human side of a machine process, make fast local decisions, and provide useful data to the larger system. With more testing and stronger integration, this type of approach could become a practical add-on for robotics labs, flexible manufacturing cells, training environments, and small automation projects.

G. References

Google AI Edge. MediaPipe Pose Landmarker for Python. Used for the pose/person detection part of the project.

https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker/python

Google AI Edge. MediaPipe Pose Landmarker Python example notebook. Example-code source for setting up and running Pose Landmarker in Python.

https://colab.research.google.com/github/googlesamples/mediapipe/blob/main/examples/pose_landmarker/python/%5BMediaPipe_Python_Tasks%5D_Pose_Landmarker.ipynb

Google AI Edge. MediaPipe Hand Landmarker for Python. Used for the hand landmark stage for the custom .pkl gesture model.

https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python

Google AI Edge. MediaPipe Gesture Recognizer for Python. Used for the .task gesture model and STOP/GO gesture recognition path.

https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer/python

Google AI Edge. MediaPipe Gesture Recognizer sample. Edge-device example that supported camera input, continuous inference, and FPS-style runtime logic.

https://github.com/google-ai-edge/mediapipe-samples/blob/main/examples/gesture_recognizer/raspberry_pi/recognize.py

OpenCV. Getting Started with Videos. Used for local webcam capture, frame reading, display, and user interface elements.

https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html